Lab Report: Fine-tuning a Transformer for Patent Phrase Matching

Kaggle: https://www.kaggle.com/code/muskaansinghla/patent-phrase-final

Task 1: Fine-tuned baseline

I ran the starter notebook as given (DeBERTa-v3-small cross-encoder, 3 epochs, LR 2e-5) and got a **validation Pearson of 0.8201** after **5.6 minutes** (337.9 s) of training on a Kaggle GPU. Out of curiosity, I ran the exact same notebook a second time, same seed and all, and this time it landed at **0.8140** (5.5 min). So even with nothing changed, the score drifts by roughly 0.006 from one run to the next, which ended up mattering later on.

Epoch	Val Pearson (run 1)	Val Pearson (run 2)
1	0.7961	0.7870
2	0.8173	0.8098
3	0.8201	0.8140

Why the split holds out whole anchors: each anchor shows up paired with lots of different targets. If the split were done on random rows, the same anchor would end up in both train and validation, and the model would effectively have seen it already. That would inflate the validation score and tell us nothing about how the model handles anchors it has never met.

Note: I had to swap `warmup_ratio=0.1` for the equivalent 10% warmup written as `warmup_steps`, since the installed version of transformers no longer accepts `warmup_ratio`. Apart from that, the starter code is untouched.

Task 2: Baseline without fine-tuning

Fine-tuning gave a gain of roughly **0.23** in Pearson compared to using a pretrained model straight out of the box.

Model	Trained on this data?	Val Pearson
Bi-encoder: all-MiniLM-L6-v2, cosine similarity of anchor and target	No	0.5814
Cross-encoder: DeBERTa-v3-small (starter, same split)	Yes	0.8140

Bi-encoder vs cross-encoder: a bi-encoder encodes the anchor and the target separately into two vectors and only compares them at the end, using cosine similarity. The two phrases never actually interact inside the model. A cross-encoder, on the other hand, takes both phrases plus the context as a single input, so attention can line them up word by word.

What fine-tuning added: the pretrained bi-encoder only has a general sense of meaning and knows nothing about the patent context. The fine-tuned cross-encoder learns what "similar" actually means for this dataset and compares the two phrases directly, and that is where the big jump comes from.

Task 3: Controlled experiments

Neither experiment gave a clear win over the baseline. In each run I changed exactly one setting from the starter and kept the same split and seed.

Setting changed	Val Pearson	Train time (min)	Interpretation
None (baseline)	0.8140 (0.8201 in another run)	5.5	Reference point. Two identical runs differ by 0.006, so I treat that as the noise floor.
No context (target only)	0.8137	5.1	Barely moves; the one-word section name doesn't add much.
Learning rate 2e-5 → 5e-5	0.8205	5.4	+0.0065, but that's inside the run-to-run noise, so I wouldn't call it a real gain.

Conclusion: dropping the context made no real difference, which suggests the CPC section name isn't giving the model much extra information here. The higher learning rate looked a little better, but since the baseline itself moved by 0.006 between two runs with the same seed, that gap could easily be noise. I'd need a few more seeds to say for sure. One thing I did notice: with LR 5e-5 the training loss dropped much further (0.030 vs 0.041) while the validation loss went up in epoch 3, which looks like the start of overfitting.

Task 4: Error analysis

Everything in this section comes from the baseline model's `val_predictions.csv` (val Pearson 0.8140).

1. Pearson per CPC section

Section	Field	Rows	Pearson
B	Performing operations; transporting	1926	0.8912
F	Mechanical engineering	968	0.8410
E	Fixed constructions	373	0.8061
H	Electricity	933	0.7957
A	Human necessities	943	0.7797
D	Textiles; paper	199	0.7765
C	Chemistry; metallurgy	1177	0.7561

Section	Field	Rows	Pearson
G	Physics	1040	0.7023

The model struggles most with **G (Physics)** and **C (Chemistry)**. Physics phrases tend to be short and generic ("imaging axis", for example), so a small change in wording can flip the meaning completely. Chemistry is full of specialised names and abbreviations the model probably hasn't seen much of. D only has 199 rows, so I wouldn't read too much into its score.

2. Mean prediction for each true score

True score	0.00	0.25	0.50	0.75	1.00
Mean prediction	0.118	0.310	0.499	0.676	1.029
Rows	1522	2465	2475	832	265

Yes, there's a clear pull toward the middle. Pairs labelled 0 get 0.118 on average (too high), and pairs labelled 0.75 get 0.676 (too low). Since most labels are 0.25 or 0.5, the loss naturally nudges predictions toward those common values. The 1.0 pairs don't get pulled in, probably because they're near-identical phrases and easy to recognise.

3. Ten largest errors

Anchor	Target	Context	True	Pred
punch face	face punch	B05	0.00	0.878
boom hydraulic cylinder	boom a hydraulic cylinder	E02	0.00	0.838
imaging axis	imaging of axis	G02	0.00	0.765
end pins	wood pin	G10	0.75	-0.011
activating position	active region	G06	0.00	0.685
imaging axis	image of axis	G02	0.00	0.675
breech mechanism	breech loading naval gun	B29	0.75	0.080
return structure	return to structure	D01	0.00	0.668
travele in direction	travele in response	F42	0.00	0.656
boom hydraulic cylinder	boom a hydraulic	E02	0.00	0.619

Failure types:

1. **Same words, different meaning (8 of 10).** The target uses nearly the same words, just reordered or with a small word slipped in ("punch face" vs "face punch", "return structure" vs "return to structure"). The model latches onto the overlapping words and predicts high, even though the label is 0.

2. **Related meaning, few shared words (2 of 10).** Pairs like "end pins" vs "wood pin" or "breech mechanism" vs "breech loading naval gun" are related within their field but have very little text in common. Getting these right needs domain knowledge the model doesn't have, so it predicts close to 0.

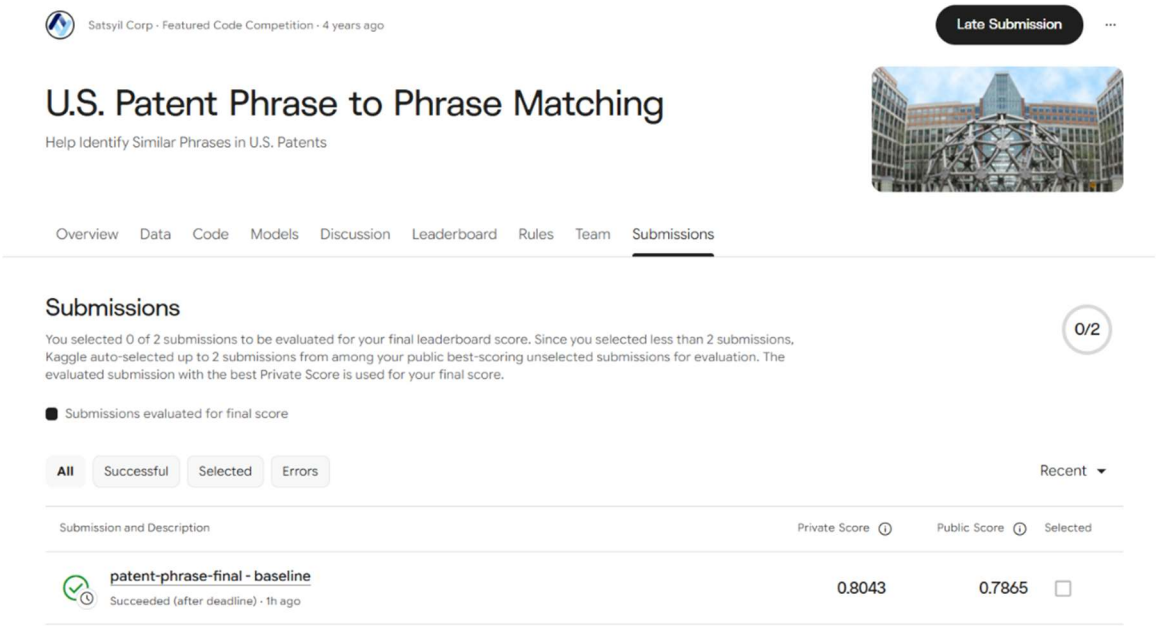
Debatable labels: "imaging axis" vs "imaging of axis" and "boom hydraulic cylinder" vs "boom a hydraulic cylinder" are labelled 0, but honestly they look very close in meaning to me. The model's high score seems fair here, so some of these "errors" are probably label noise.

Task 5: Late submission and leaderboard scores

I went with the **starter baseline** as my final model, since none of the experiments beat it on validation by more than the run-to-run noise. I made **one** late submission.

Submission	Validation	Public	Private
patent-phrase-final (baseline)	0.8201	0.7865	0.8043

Figure 1 (below): Kaggle Submissions tab showing this submission (Private 0.8043, Public 0.7865).



Comparison: the scores are in the same ballpark but not identical. Private is 0.016 below validation, and public is 0.034 below. A few likely reasons:

- The hidden test set probably has a different mix of anchors and CPC sections than my validation split.
- The public split is smaller, so its score is noisier.
- My own validation score already wobbles by about 0.006 between identical runs.

References

1. Aslanyan, G., & Wetherbee, I. (2022). *Patents Phrase to Phrase Semantic Matching Dataset.* arXiv:2208.01171
2. Google Research (2022). *Announcing the Patent Phrase Similarity Dataset.* Blog post
3. Kaggle. *U.S. Patent Phrase to Phrase Matching* competition. Competition page
4. He, P., Gao, J., & Chen, W. (2021). *DeBERTaV3.* arXiv:2111.09543
5. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT.* EMNLP 2019. arXiv:1908.10084
6. Hugging Face model card: sentence-transformers/all-MiniLM-L6-v2 (used in Task 2).
7. Hugging Face documentation: Trainer, Sentence Transformers.
8. Course starter notebook `patent_phrase_starter.ipynb` and `download_weights.ipynb`.